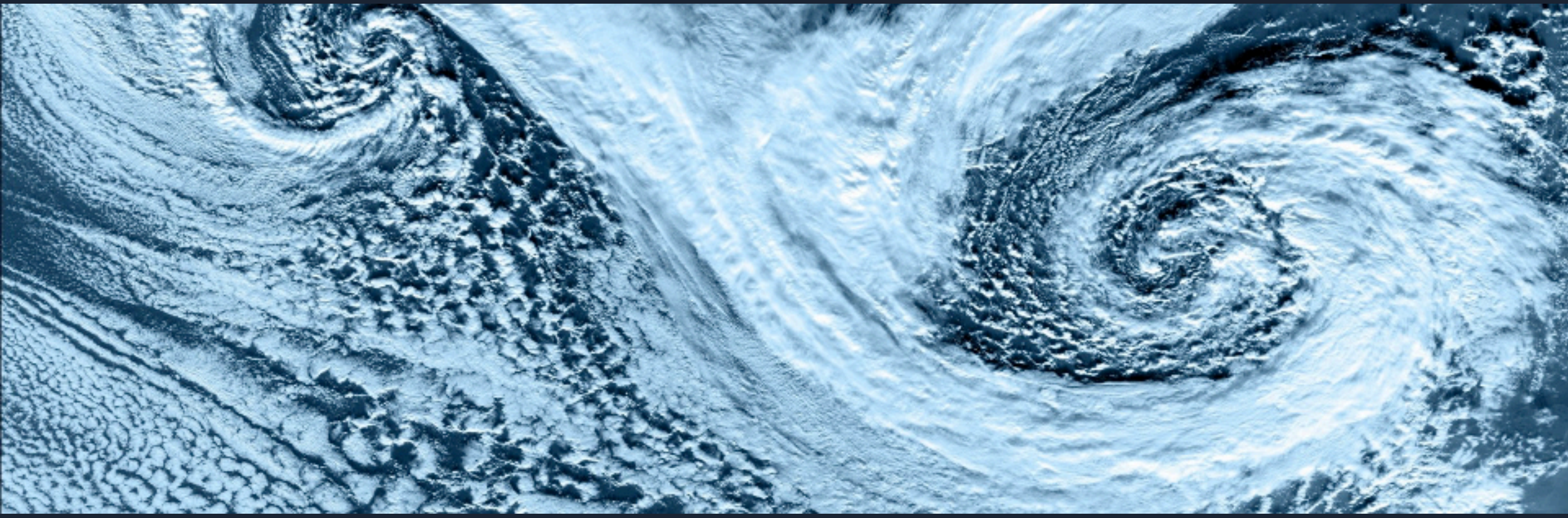
Git: Beyond the Basics

C2SM Git Courses · 8 October 2026

Michael Jähn, Mikael Stellio, Alitzel Macías Infante

C2SM
Center for Climate
Systems Modeling



Outline

C2SM
Center for Climate
Systems Modeling

Part 1: Your Git Toolbox

- Examining history: `log`, `blame`, `diff`, `show`
- Nesting repositories with submodules
- Ignoring files
- `git cherry-pick` and `git rebase`
- `git stash` and `git worktree`
- Custom Git hooks
- Large files with `git lfs`
- Exercises 1 – 7

Part 2: Working Together on GitHub

- Why a shared workflow
- A real example: the C2SM User Landing Page
- Issues, forks and branches
- Pull requests, checks and review
- Keeping your fork in sync
- GitHub and GitLab side by side
- Exercise 8

> You only need the **Git basics** for this course: `add`, `commit`, `push`, `pull`, `branch`.

Schedule

09:30 – 09:40 Welcome and overview

09:40 – 10:50 Part 1 · history, submodules, .gitignore → Exercises 1 – 3

10:50 – 11:10 Coffee break ☕

11:10 – 11:50 Part 1 · moving, parallel work → Exercises 4 – 5

11:50 – 12:30 Part 1 · hooks and large files → Exercises 6 – 7

12:30 – 13:30 Lunch break 🍽️

13:30 – 14:30 Part 2 · slides and live demonstration

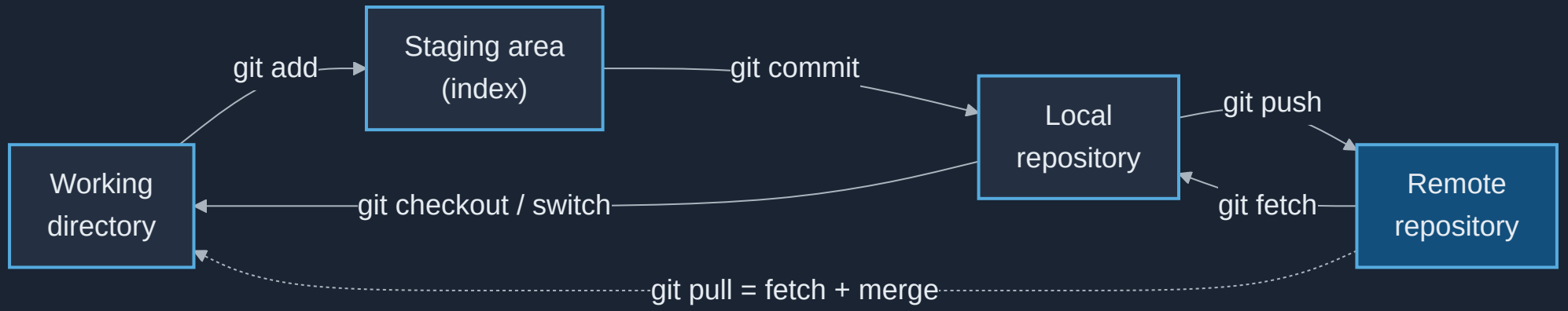
14:30 – 15:00 Part 2 · Exercise 8 and wrap-up

C2SM
Center for Climate Systems Modeling

Part 1:

Your Git Toolbox

Recap: Typical Git Workflow



- Everything up to `git commit` happens **on your machine**
- Only `git push` and `git fetch` talk to the server

Examining History: Four Commands

`git log`

which commits exist, by whom, and in what order

`git blame`

which commit last changed each **line** of a file

`git diff`

what changed between two points in the repository

`git show`

a single commit: message **and** its diff

Each takes many options. Exercise 1 explores the ones worth remembering.

`git log`: Shaping and Filtering

C2SM Center for Climate Systems Modeling

Option	What it does
`--oneline`	one line per commit
`--graph --decorate --all`	the branch structure of the whole repository
`--stat`	which files each commit touched
`-3`	only the last three commits
`--author="Lauber"`	only commits by a given author
`-- path/to/file`	only commits touching that file
`-S "Have fun!"`	commits that **add or remove** that text
`-G "regex"`	commits whose diff matches a regular expression

Found a combination you like? Save it: `git config --global alias.lg "log --oneline --graph --decorate --all"`

`git diff`: Choosing Two Points

`git diff`

working directory vs. staging area

`git diff --staged`

staging area vs. last commit (what `git commit` would record)

`git diff main my-branch`

one branch against another

`git diff HEAD~10 HEAD~5 -- README.md`

two commits, restricted to one file

> Hard to read in a terminal? Use `git difftool --tool-help` to see what your system offers, or the web interface: add `/compare` to any GitHub repository URL.

Part 1 · Exercise 1

`git log`, `git blame`, `git diff` and `git show`

Command	What it does
`git log --oneline --graph --decorate --all`	branch structure at a glance
`git log --author="name"` / `-- path` / `-S "text"`	filter commits
`git blame <file>`	who last touched each line
`git diff` / `git diff --staged`	unstaged vs. staged changes
`git show <commit>` / `git show <commit>:<path>`	inspect a commit / an old file version

Where you work: the `git-course` repository itself - we examine its real history.
All exercises: https://github.com/C2SM/git-course/tree/main/advanced

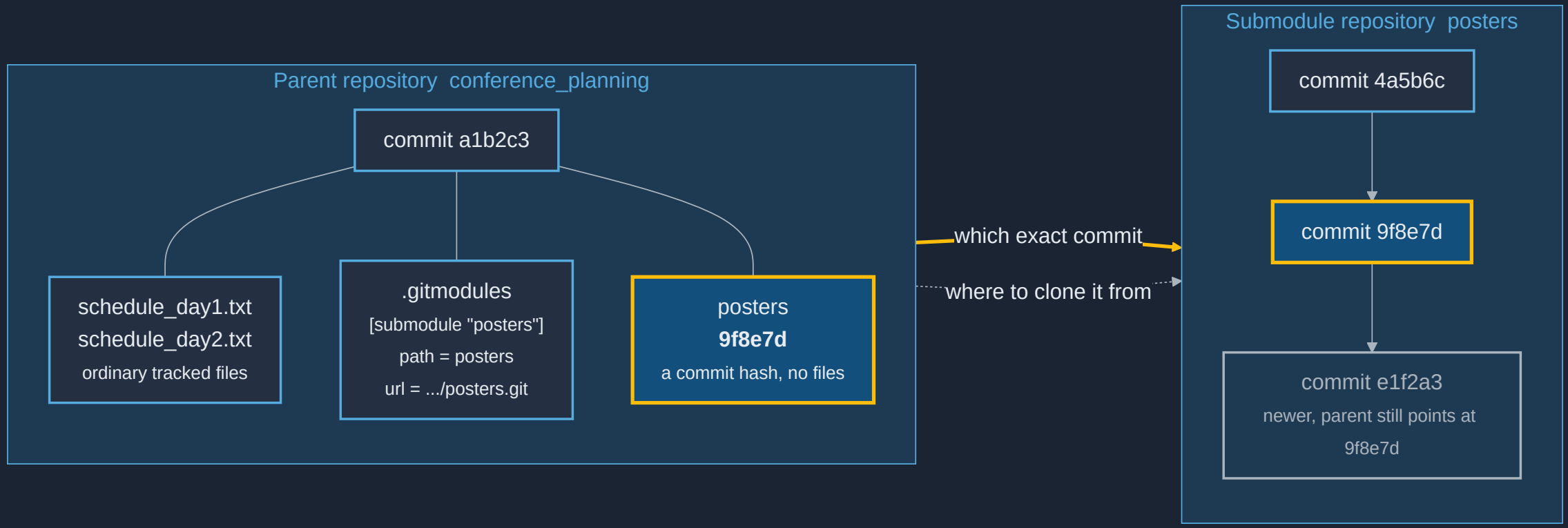
Nesting Repositories

C2SM Center for Climate Systems Modeling

- **Why?**
  - **Modularity** - break a large project into pieces that stand alone
  - **Independent history** - each piece keeps its own commits and releases
  - **Collaboration** - separate teams own separate pieces

- **Two mechanisms**
  - **Submodules** - Git's built-in approach, and the one C2SM models use
  - **Subtrees** - an alternative that copies content into the parent instead

A Submodule Is a Pointer to One Commit



The parent repository records **one exact commit** of the submodule, not "the latest".

Working With Submodules

C2SM
Center for Climate
Systems Modeling

`git submodule add <url> <path>`

adds the submodule and creates `.gitmodules`

`git clone --recurse-submodules`

clones the parent **and** fills in the submodules

`git submodule update --init`

fills them in after an ordinary `git clone`

`git submodule update --remote --merge`

moves the pointer forward to the submodule's latest commit

⚠ A plain `git clone` gives you **empty** submodule directories. This surprises everybody once.

Submodules: Pros and Cons

C2SM
Center for Climate
Systems Modeling

Pros

- Built into Git, nothing to install
- The parent records an exact, reproducible commit
- The submodule stays a normal repository
- Same submodule can be reused across multiple parent repos

Cons

- Every clone needs an extra step
- Easy to commit a pointer you never pushed
- Detached `HEAD` inside the submodule confuses people
- Branch operations do not recurse by default

> Some C2SM models and tools use submodules. Knowing the two or three key commands takes most of the pain away.

Part 1 · Exercise 2

C2SM Center for Climate Systems Modeling

`git submodule`

Command	What it does
`git submodule add <url> <path>`	add a submodule
`git submodule status`	which commit is currently recorded
`git submodule update --init --recursive`	fill in submodules after a plain clone
`git submodule update --remote --merge`	move the pointer to the submodule's latest commit
`git clone --recurse-submodules <url>`	clone parent and submodules in one step

Where you work: `advanced_git/conference_submodule`

Everything stays local - the helper script builds a small stand-in repository to point the submodule at.

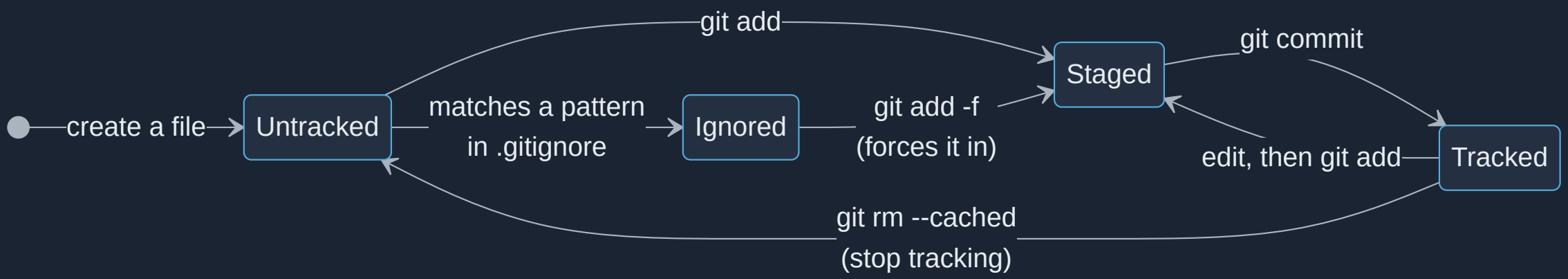
The `.gitignore` file

- Tell Git to leave alone what should never be committed
- Build products, binaries, editor droppings, secrets, large data
- Patterns live in a `.gitignore` file - commit it, so the everyone shares the same rules

```
*~
*.exe
netcdf-*
build/
!build/keep_this.sh
```

- Patterns are matched per directory; `!` re-includes something
- `git check-ignore -v <file>` tells you **which line** ignored a file

Ignoring Files: the States



⚠ `.gitignore` only affects **untracked** files. A file already committed keeps being tracked until you run `git rm --cached <file>`.

`.gitkeep`

- Git tracks **files**, never directories
- An empty directory simply will not be committed
- Convention: put an empty `.gitkeep` file inside it and commit that

> **Note:** `.gitkeep` is a community convention, not a Git feature. The name has no special meaning - any file would do.

Part 1 · Exercise 3

`.gitignore` and `.gitkeep`

Command	What it does
`*~` , `build/` , `!keep.sh`	pattern, directory, re-include in `.gitignore`
`git check-ignore -v <file>`	which rule and line ignores a file
`git rm --cached <file>`	untrack a file already committed
`git add path/.gitkeep`	keep an otherwise empty directory
`git config --global core.excludesFile <file>`	personal, machine-wide ignore rules

Where you work: `advanced_git/conference_planning`

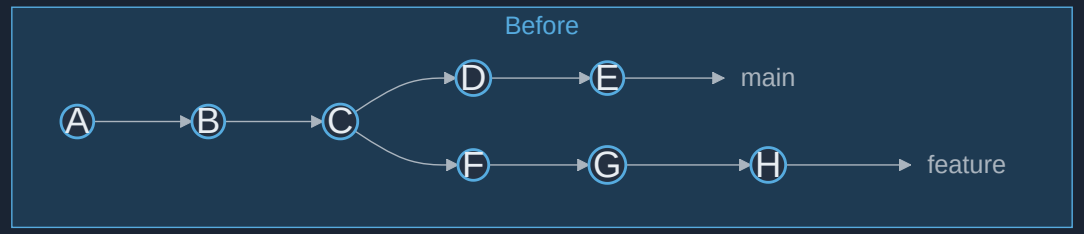
Stuck? `reset_advanced_repo` gives you a clean start at any time.

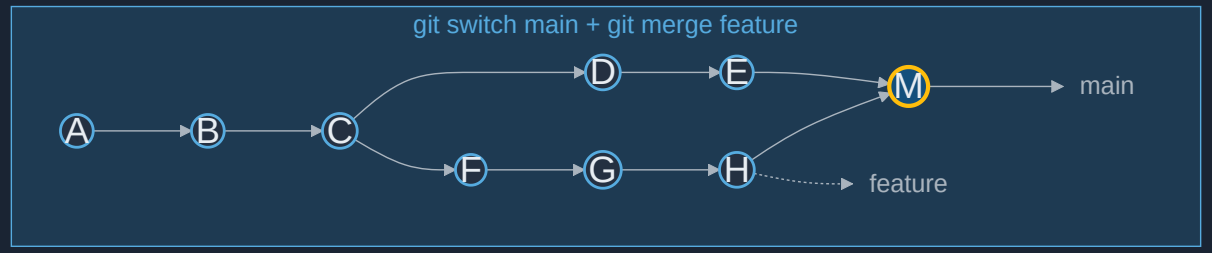
`git cherry-pick`: One Commit, Copied

- Takes a single commit and replays it on your current branch
- The copy gets a **new commit ID** - same content, different identity

⚠ Do not merge the branch you picked from later: the change would arrive twice.

`git rebase`: an Alternative to `git merge`





Merge joins the lines with `M` - **rebase** replays `F G H` on top of `E` as new commits `F' G' H'`

> ⚠ Rebasing **rewrites history**. Never rebase a branch other people have already pulled.

Part 1 · Exercise 4

C2SM Center for Climate Systems Modeling

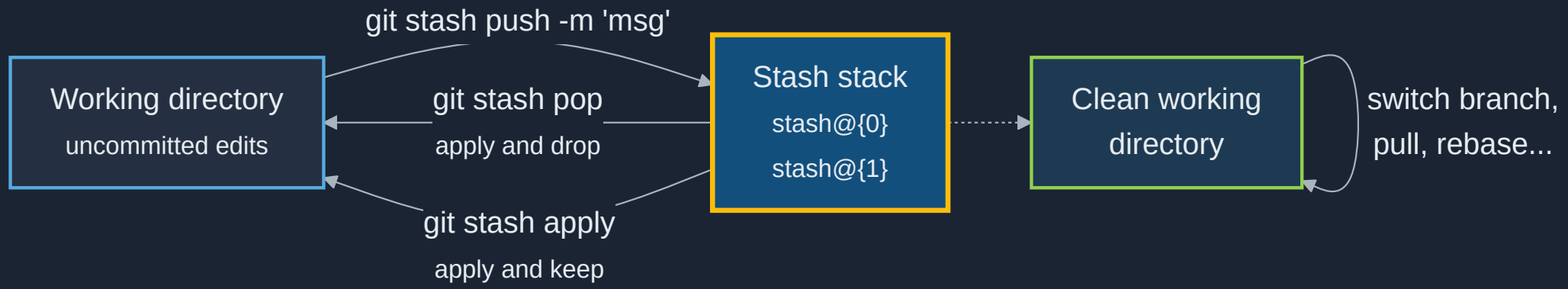
`git cherry-pick` and `git rebase`

Command	What it does
`git switch -c <branch>`	create and switch to a new branch
`git log --oneline --graph --decorate --all`	see the shape of the history
`git cherry-pick <commit-id>`	replay one commit onto the current branch
`git cherry-pick --abort`	back out of a conflicted cherry-pick
`git rebase <upstream> <branch>`	replay `<branch>`'s commits onto `<upstream>`
`git merge <branch>`	join two lines of history with a merge commit

Where you work: `advanced_git/conference_planning`

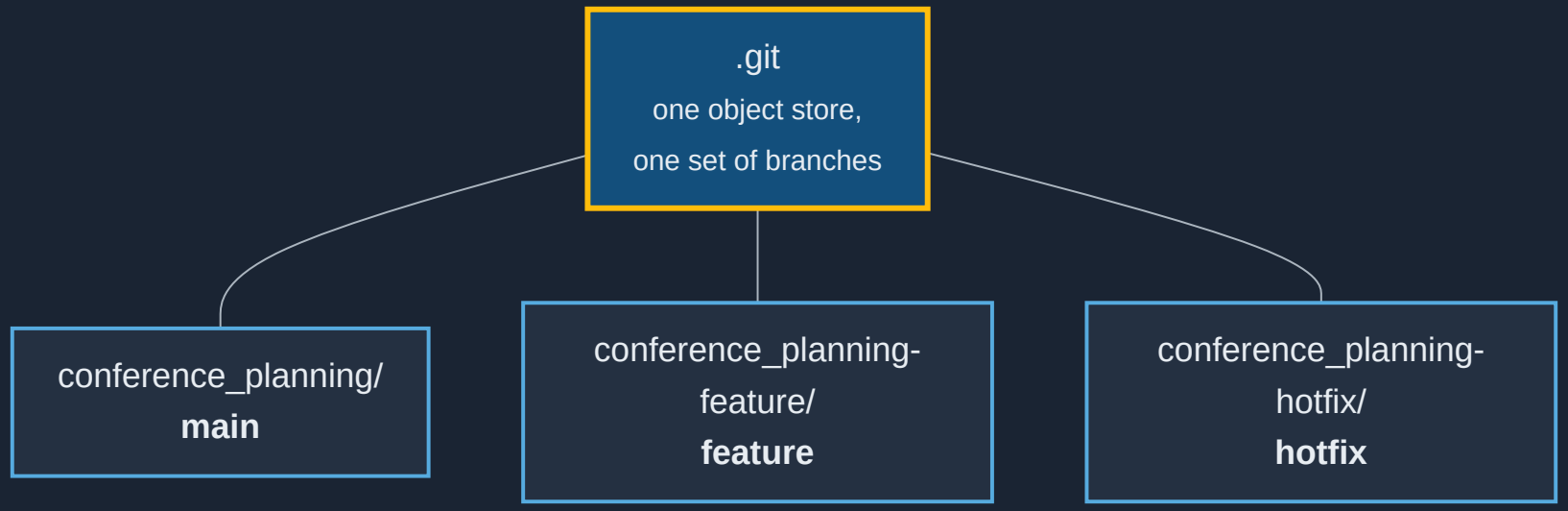
Stuck? `reset_advanced_repo` gives you a clean start at any time.

`git stash`: Park Your Work



- For when you must switch branch or pull, but are not ready to commit
- `git stash push -m "message"`, then `git stash list`, then `git stash pop`
- `-u` also stashes untracked files
- Stashes are **local only** - they never reach a remote

`git worktree`: Several Branches at Once



- Multiple working directories sharing **one** `.git`
- Cheaper than a second clone, and the configuration stays in one place
- Especially useful when switching branches means recompiling

```
git worktree add ../conference_planning-feature feature
```

Part 1 · Exercise 5

`git stash` and `git worktree`

Command	What it does
`git stash push -m "msg" / -u`	park changes, `-u` includes untracked files
`git stash list` / `git stash show -p stash@{0}`	see stashes and what one contains
`git stash pop / git stash apply`	restore, dropping / keeping the stash entry
`git worktree add -b <branch> <path>`	new working directory on a new branch
`git worktree list / remove <path> / prune`	manage worktrees

Where you work: `advanced_git/conference_planning`, plus the worktree it creates next to it.

C2SM
Center for Climate Systems Modeling

Coffee Break

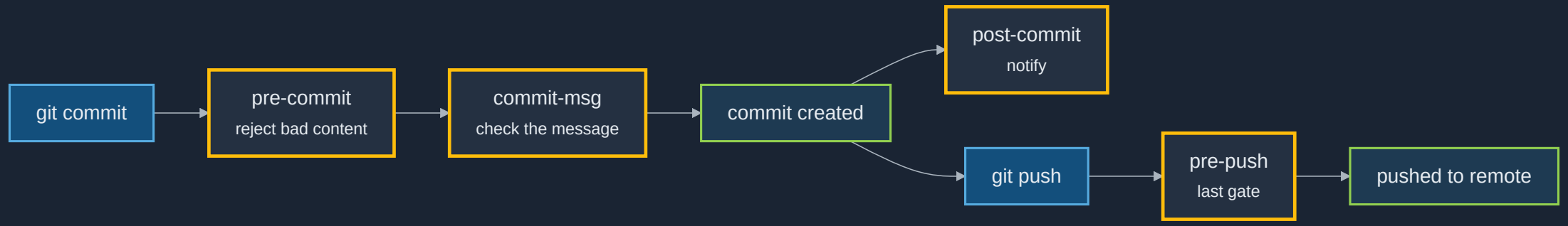
Custom Git Hooks

C2SM Center for Climate Systems Modeling

- Scripts Git runs automatically when a certain event happens
- Live in `.git/hooks`, named after their event, must be **executable**
- A non-zero exit status from a `pre-` hook **cancels** the operation
- `.git/hooks` is **not** part of the repository, so hooks are not shared by cloning
- Git ships `.sample` files for every hook - rename one to activate it

> Want hooks that everybody gets? Commit them to a folder and point Git at it with `git config core.hooksPath <folder>`, or use the pre-commit framework.

When Each Hook Fires



Typical uses: reject trailing whitespace, run a formatter, enforce a commit-message format, block commits of secrets, run fast tests before a push.

Part 1 · Exercise 6

C2SM Center for Climate Systems Modeling

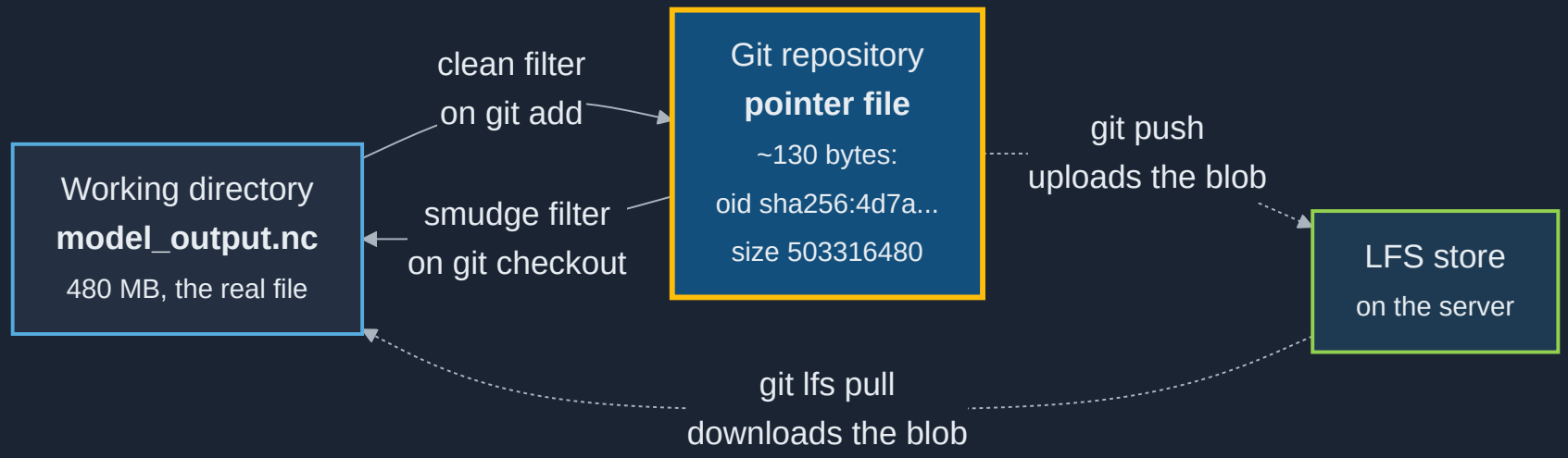
Custom Git hooks

Command	What it does
`.git/hooks/pre-commit`	hook file, named after the event, no extension
`chmod +x .git/hooks/pre-commit`	make it executable - required, or it's ignored
`git commit --no-verify`	skip hooks for this one commit
`git config core.hooksPath <folder>`	use a committed, shared hooks folder instead

Where you work: `advanced_git/conference_planning`
Everything happens in its `.git/hooks` directory.

Large Files: `git lfs`

- Git stores a **full copy of every version** of every file
- That is perfect for text and terrible for a 500 MB NetCDF file
- Git Large File Storage keeps a tiny **pointer** in the repository and the real bytes elsewhere



Using `git lfs`

C2SM
Center for Climate
Systems Modeling

`git lfs install`

once per machine: registers the filters

`git lfs track "*.nc"`

records the pattern in `.gitattributes` - **commit that file**

`git lfs ls-files`

which files are handled by LFS

⚠ LFS is not free: hosts put quotas on storage and bandwidth, and rewriting LFS history is painful. Before reaching for it, ask whether the data belongs in a data archive instead.

Part 1 · Exercise 7

C2SM Center for Climate Systems Modeling

`git lfs`

Command	What it does
`git lfs install`	once per machine: registers the filters
`git lfs track "*.nc"`	record a pattern in `.gitattributes` - commit that file
`git lfs ls-files` / `git lfs status`	which files are handled by LFS
`git cat-file -p HEAD:<path>`	see the raw pointer Git actually stores

Where you work: `advanced_git/conference_planning`
Entirely local - no remote and no LFS quota needed.

C2SM
Center for Climate Systems Modeling

Lunch Break

C2SM
Center for Climate Systems Modeling

Part 2:

Working Together on GitHub

Why a Shared Workflow?

Three problems appear the moment a second person joins:

- **Access** - nobody can push to a protected `main`, so "just push it" is not an option
- **Traceability** - a change needs a visible record of *why*, not only *what*
- **Review** - somebody should look at a change **before** it reaches everyone else

A web interface solves all three with the same object: the **pull request**.

> The Git commands you already know do not change. What follows is a convention layered on top of them.

A Real Example: the C2SM User Landing Page

https://github.com/C2SM/c2sm.github.io → https://c2sm.github.io

- Documentation for models, tools, datasets and HPC systems used across C2SM
- Written as plain Markdown, built into a website automatically
- Maintained by the core team, but **anyone in the group can contribute**
- Every pull request gets a **preview website** before anything is merged

We will walk through a real change to this repository, then you will practice the same workflow on https://github.com/C2SM/c2sm-git-example.

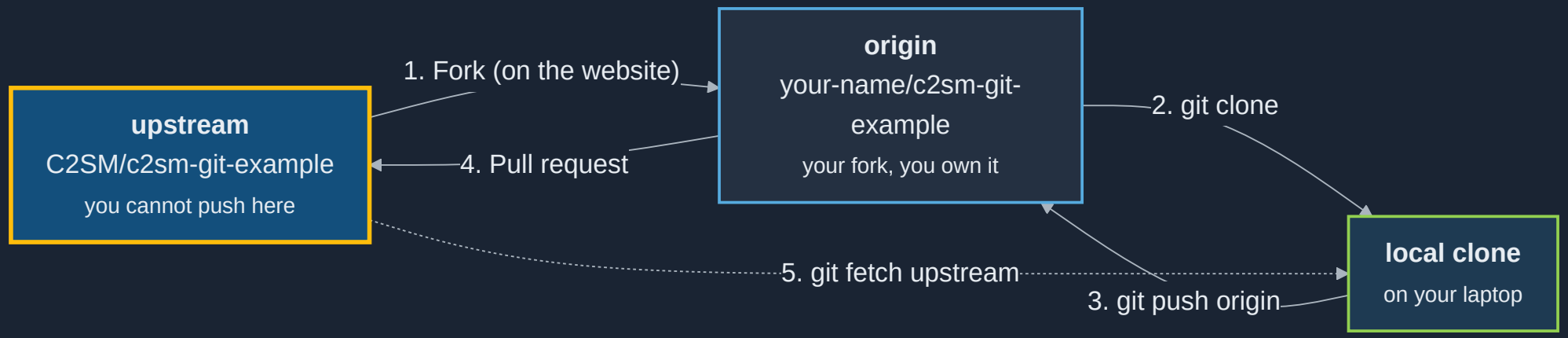
It Starts With an Issue

- An issue describes **what is wrong or missing, and why** - before any code is written
- It is the place to agree on an approach before someone spends a day on it
- Labels, assignees and milestones make a backlog searchable months later

- Write `Fixes #12` in a pull request description and GitHub **closes issue 12 automatically** when it merges

> A good issue is reproducible: what you did, what you expected, what happened instead.

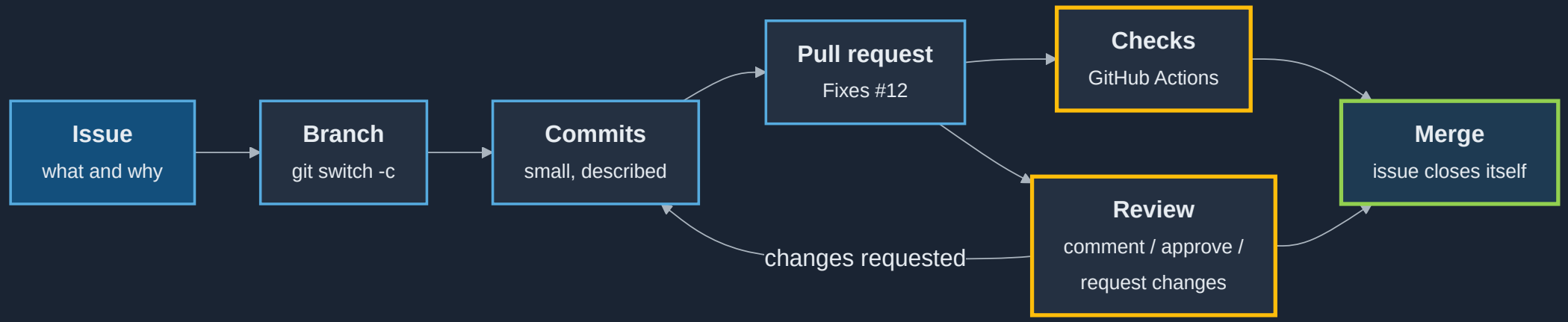
Fork and Branch



Fork when you cannot push to the original. **Branch** when you can. Either way the change arrives as a pull request.

The Pull Request

- A request to merge one branch into another, plus the conversation around it
- The **description** is the lasting record - say why, not just what
- Open it **early** as a draft to show work in progress
- Keep it small: a reviewer reads 200 lines carefully and 2000 lines not at all



Automated Checks

C2SM Center for Climate Systems Modeling

- GitHub Actions run on every pull request, before a human looks at it
- On the landing page repository they check that Markdown links resolve and that the site still builds
- A failing check blocks the merge, so broken changes never reach `main`

- The same workflow deploys a **preview of the website for that pull request**: `https://c2sm.github.io/pr-preview/pr-<number>/`

> Reviewers can look at the rendered page, not just the diff. This is the single biggest reason the landing-page workflow works well.

Code Review

C2SM
Center for Climate
Systems Modeling

As the author

- Small, focused changes
- Explain the reasoning
- Reply to every comment
- Push fixes as new commits

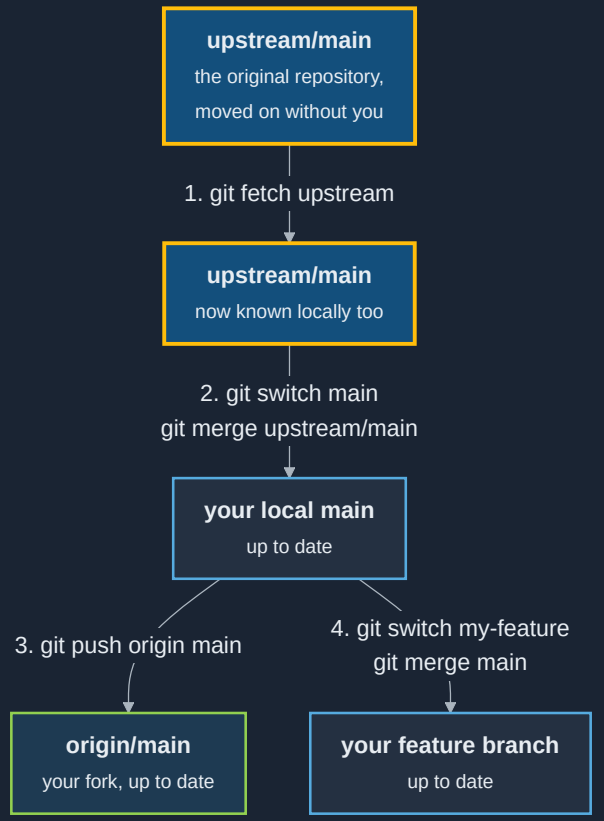
As the reviewer

- Comment, approve, or request changes
- Use **suggestions** - the author can apply them with one click
- Ask questions instead of issuing orders
- Approve when it is good enough, not perfect

> Review is about the change, never the person. "This function could be clearer" beats "you wrote this badly".

Merging, and Staying in Sync

- **Merge commit** - keeps every commit and records the merge
- **Squash** - collapses the branch into one tidy commit (a common default)
- **Rebase** - replays commits with no merge commit
- Delete the branch afterwards; the pull request keeps the history



GitHub and GitLab Side by Side

C2SM works on both `github.com` and `gitlab.ethz.ch`. The **local Git commands are identical** - only the website and the CI file differ.

Concept	GitHub	GitLab
Proposed change	Pull request (PR)	**Merge request (MR)**
Close an issue automatically	`Fixes #12` / `Closes #12`	`Closes #12`
CI configuration	`.github/workflows/*.yml`	`.gitlab-ci.yml`
Sign-off on a change	Approve / request changes	**Approvals**, a required count
Static site hosting	GitHub Pages	GitLab Pages
Preview of a change	PR preview via an Action	Review Apps
Ownership rules	`CODEOWNERS`	`CODEOWNERS`
Update a fork	"Sync fork" button or `upstream` remote	`upstream` remote
Namespaces	User / organization	User / **group**, nestable

C2SM
Center for Climate Systems Modeling

Live Demonstration

https://github.com/C2SM/c2sm.github.io

Part 2 · Exercise 8

C2SM Center for Climate Systems Modeling

An issue, a fork, a pull request and a review

Command	What it does
`git remote add upstream <url>` / `git remote -v`	name the original repository, list remotes
`git switch -c <branch>`	start your change on its own branch
`git push -u origin <branch>`	send the branch to your fork
`git fetch upstream` / `git merge upstream/main`	bring your fork's `main` up to date
`Fixes #<n>`	in the PR description, closes the issue on merge

> **Where you work:** https://github.com/C2SM/c2sm-git-example in the browser, plus a clone of your fork anywhere outside `advanced_git`.

You will review each other's pull requests, so **work in pairs**.

Useful Tools

C2SM Center for Climate Systems Modeling

In your editor

- VS Code - built-in Git, GitLens
- magit (Emacs)
- vim-fugitive (Vim)
- JetBrains IDEs

Better diffs

- delta
- difftastic

In the terminal

- gh - pull requests and issues from the shell
- lazygit - terminal interface
- tig - history browser

Graphical

- GitHub Desktop
- Git GUIs - a long list

Where to Look Things Up

- The Git book, free and genuinely good: https://git-scm.com/book
- Git reference: https://git-scm.com/docs
- GitHub documentation: https://docs.github.com
- GitHub cheat sheet: https://education.github.com/git-cheat-sheet-education.pdf
- When something has gone wrong: https://dangitgit.com
- Topics beyond this course: Expert_Topics.md

⚠ Language models are often useful for Git because the documentation is so good. They also invent flags that do not exist. Check `git help <command>` before running anything you do not recognize - especially anything with `--force`.

References

- Chacon, S. & Straub, B. *Pro Git*, 2nd ed. Apress, 2014. https://git-scm.com/book
- Git Project. *Git Reference Documentation* - `git-log`, `git-diff`, `git-submodule`, `git-cherry-pick`, `git-rebase`, `git-stash`, `git-worktree`, `githooks`. https://git-scm.com/docs
- Git LFS Project. *Git Large File Storage Documentation*. https://git-lfs.com
- pre-commit. *A Framework for Managing Multi-Language Pre-Commit Hooks*. https://pre-commit.com
- GitHub, Inc. *GitHub Docs*. https://docs.github.com
- GitLab B.V. *GitLab Docs*. https://docs.gitlab.com
- C2SM. *c2sm.github.io* - the User Landing Page used as the worked example. https://github.com/C2SM/c2sm.github.io

C2SM
Center for Climate
Systems Modeling

Questions? Comments?